

```

1  (*
2                                     CS51 Lab 7
3                                     Modules and Abstract Data Types
4  *)
5
6  (*
7                                     SOLUTION
8  *)
9
10 (*=====
11 Readings:
12
13     This lab builds on material from Chapters 12-12.4 of the textbook
14     <http://book.cs51.io>, which should be read before the lab session.
15
16 Objective: This lab practices concepts of modules, including files
17 as modules, signatures, and polymorphic abstract data types.
18
19 There are three total parts to this lab. Please refer to the following
20 files to complete all exercises:
21
22 -> lab7_part1.ml -- Part 1: Implementing modules (this file)
23     lab7_part2.ml -- Part 2: Files as modules
24     lab7_part3.ml -- Part 3: Polymorphic abstract types and
25                        Interfaces as abstraction barriers
26 =====*)
27
28 (*=====
29 Part 1: Implementing Modules
30
31 *Modules* are a way to package together and encapsulate types and values
32 (including functions) into a single discrete unit.
33
34 By applying a *signature* to a module, we guarantee that the module
35 implements at least the values and functions defined within it. The
36 module may also implement more as well, for internal use, but only those
37 specified in the signature will be exposed and available outside the
38 module definition. This form of abstraction, information hiding,
39 implements the edict of compartmentalization.
40
41 In this part, you'll revisit the "weather" example from lab 6 part
42 1. Recall that in that lab, you defined some algebraic data types for
43 representing aspects of the season and weather, along with some
44 functions to extract the precipitation amount and to generate a string
45 description of the weather. We can define a module signature for the
46 types and functions from that lab as follows:
47 *)
48
49 module type WEATHER = sig
50     type season = Spring | Summer | Autumn | Winter
51
52     type condition =

```

```

53 | Sunny
54 | Rainy of int (* precipitation in mm *)
55 | Snowy of int (* precipitation in mm *)
56
57 type weather_status = {season : season; condition : condition}
58
59 val describe_weather : weather_status -> string
60 val precipitation_amount : condition -> int
61 end ;;
62
63 (* Notice that we've left out the function `season_to_string` from the
64 signature, since it was really just a helper function for
65 `describe_weather`. There's no reason that users of the module should
66 need to use this function. *)
67
68 (*.....
69 Exercise 1A: Complete the implementation of a module called `Weather`
70 that satisfies the signature above. Feel free to make use of your
71 solution or the staff solution for lab 6 part 1.
72 .....*)
73
74 module Weather : WEATHER = struct
75   type season = Spring | Summer | Autumn | Winter
76
77   type condition =
78     | Sunny
79     | Rainy of int
80     | Snowy of int
81
82   type weather_status = {season : season; condition : condition}
83
84   (* We still make use of the `season_to_string` helper function. It
85      won't be exposed in the interface. *)
86   let season_to_string (season : season) : string =
87     match season with
88     | Spring -> "spring"
89     | Summer -> "summer"
90     | Autumn -> "autumn"
91     | Winter -> "winter"
92
93   let precipitation_amount (condition : condition) : int =
94     match condition with
95     | Sunny -> 0
96     | Rainy amount | Snowy amount -> amount
97
98   let describe_weather { season; condition } =
99     let season_str = season_to_string season in
100     let amount = precipitation_amount condition in
101     let common_str = " in " ^ season_str
102                   ^ ". Precipitation: "
103                   ^ string_of_int amount ^ " mm." in
104     match condition with
105     | Sunny -> "It's a sunny " ^ season_str ^ " day."

```

```

106 | Rainy _ -> "It's raining" ^ common_str
107 | Snowy _ -> "It's snowing" ^ common_str
108 end ;;
109
110 (*.....
111 Exercise 1B: Now that you've implemented the `Weather` module, use it
112 to generate a string description of a rainy winter day with 20 mm of
113 rain. That is, define a value `example : string` that uses
114 the `Weather` module to generate its string value
115
116 "It's raining in winter. Precipitation: 20 mm."
117
118 (Use explicit module prefixes for this exercise, not global or local
119 opens.)
120 .....*)
121
122 let example =
123   Weather.describe_weather
124   {Weather.season = Weather.Winter;
125    Weather.condition = Weather.Rainy 20} ;;
126
127 (* You may find that, strictly speaking, the `Weather.` module
128 qualifiers are not needed on the field names (like `season`) and
129 value constructors (like `Sunny`). That's because OCaml may manage
130 to infer which `season` or `Sunny` you may be referring to. But
131 OCaml can't always reconstruct them, so we've included them for
132 explicitness, and to highlight the contrast in the next
133 exercise. *)
134
135 (*.....
136 Exercise 1C: Reimplement `example` from 1B above, now as
137 `example_local_open`, but using a "local open" to write your
138 computation in a more succinct manner.
139 .....*)
140
141 let example_local_open =
142   let open Weather in
143     describe_weather {season = Winter; condition = Rainy 20} ;;

```